

✅ Veille Technologique : Le BOM et la Boîte à Outils JavaScript

🎯 Objectif

Comprendre comment JavaScript interagit avec l'environnement du navigateur via le **BOM**, et comment il manipule efficacement les données grâce aux **méthodes natives** sur les chaînes, les nombres, les tableaux et les objets.

1 Le BOM : L'Environnement du Navigateur

► Définition

Le **BOM** (Browser Object Model) représente *tout l'environnement fourni par le navigateur* autour de la page Web.

Il permet à JavaScript d'interagir avec :

- la **fenêtre** (window)
- l'URL
- l'historique
- la localisation
- les informations système
- etc.

L'objet racine du BOM est **window**, et il est accessible partout dans vos scripts.

► Objets du BOM et leurs rôles

◆ 1. window.location

Rôle : gérer et interagir avec l'URL de la page.

Méthode/Propriété utile :

- `location.href` → retourne l'URL complète (ou permet d'en changer pour rediriger la page)

Exemple :

```
javascript
console.log(window.location.href); // Affiche l'URL actuelle
```

♦ 2. window.navigator

Rôle : donner des informations sur le navigateur et l'appareil.

Propriété utile :

- navigator.userAgent → renseigne sur le navigateur utilisé

Exemple :

```
javascript
console.log(navigator.userAgent);
```

2 DOM vs BOM : Quelle différence ?

DOM

Représente *le document* (HTML).

Manipule les **éléments de la page** (div, p, input...).

Racine = document.

👉 Le DOM permet de **modifier l'interface**.

👉 Le BOM permet de **contrôler le navigateur**.

BOM

Représente *le navigateur* et son environnement.

Manipule la **fenêtre**, l'URL, l'historique, les infos système...

Racine = window.

3 Méthodes de Manipulation des Types de Données

► A. Strings (chaînes de caractères)

1. str.toUpperCase()

Met toute la chaîne en majuscules.

Utile pour normaliser ou comparer des textes.

```
javascript
"bonjour".toUpperCase(); // "BONJOUR"
```

2. str.includes()

Vérifie si un mot/texte est présent dans une chaîne.

```
javascript

"Bonjour le monde".includes("monde"); // true
```

3. str.substring()

Extrait une partie de chaîne selon deux index.

```
javascript

"JavaScript".substring(4, 10); // "Script"
```

► B. Numbers (nombres)

1. Number.isInteger()

Teste si une valeur est un **entier**.

```
javascript

Number.isInteger(10); // true
```

2. num.toFixed()

Formate un nombre à un nombre fixe de décimales → retourne une *string*.

```
javascript

12.3456.toFixed(2); // "12.35"
```

3. parseInt()

Convertit une chaîne en nombre entier.

```
javascript

parseInt("25.7"); // 25
```

► C. Arrays (tableaux)

1. arr.push()

Ajoute un élément à la fin du tableau.

```
javascript

let a = [1, 2];
a.push(3); // [1, 2, 3]
```

2. arr.filter()

Crée un tableau avec les éléments qui respectent une condition.

```
javascript

[1, 2, 3, 4].filter(n => n > 2); // [3, 4]
```

3. arr.map()

Transforme chaque élément du tableau et retourne un **nouveau tableau**.

```
javascript

[1, 2, 3].map(n => n * 2); // [2, 4, 6]
```

4. arr.sort()

Trie un tableau selon un critère.

```
javascript

[3, 1, 2].sort(); // [1, 2, 3]
```

► D. Objects (objets)

1. Object.keys()

Retourne un tableau contenant toutes les clés de l'objet.

```
javascript

Object.keys({a:1, b:2}); // ["a", "b"]
```

2. Object.values()

Retourne les valeurs des propriétés.

```
javascript  
  
Object.values({a:1, b:2}); // [1, 2]
```

3. Object.entries()

Retourne des paires [clé, valeur].

```
javascript  
  
Object.entries({a:1, b:2});  
// [["a", 1], ["b", 2]]
```

Conclusion

JavaScript interagit avec le navigateur grâce au **BOM** (window, location, navigator...).

Il manipule efficacement les données via les **méthodes natives** (Strings, Numbers, Arrays, Objects).

Ces outils constituent la **boîte à outils essentielle** du développeur JavaScript moderne.